
Software Requirements Specification

for

Agente inteligente para agendar citas personales

Version 1.1

Prepared by

Group Name: Equipo 21

Ana Cristina Torres
Diego Bernal
Javier Rodríguez
Guillermo Urbina

A00831285
A01795975
A01793817
A01746645

A00831285@tec.mx
A01795975@tec.mx
A01793817@tec.mx
A01746645@tec.mx

Professor: *Dr. Alberto Aguilar González*

Course: Análisis, diseño y construcción de software

Date: 28 de septiembre 2025

Contents

CONTENTS	II
REVISIONS	III
1 INTRODUCTION	1
1.1 DOCUMENT PURPOSE	1
1.2 PRODUCT SCOPE	1
1.3 INTENDED AUDIENCE AND DOCUMENT OVERVIEW	1
1.4 DEFINITIONS, ACRONYMS AND ABBREVIATIONS	2
1.5 DOCUMENT CONVENTIONS	4
1.6 REFERENCES AND ACKNOWLEDGMENTS	4
2 OVERALL DESCRIPTION	6
2.1 PRODUCT OVERVIEW	6
2.2 PRODUCT FUNCTIONALITY	ERROR! BOOKMARK NOT DEFINED.
2.3 DESIGN AND IMPLEMENTATION CONSTRAINTS	7
2.4 ASSUMPTIONS AND DEPENDENCIES	7
3 SPECIFIC REQUIREMENTS	8
3.1 EXTERNAL INTERFACE REQUIREMENTS	8
3.2 FUNCTIONAL REQUIREMENTS	9
3.3 USE CASE MODEL	10
4 OTHER NON-FUNCTIONAL REQUIREMENTS	23
4.1 PERFORMANCE REQUIREMENTS	23
4.2 SAFETY AND SECURITY REQUIREMENTS	23
4.3 SOFTWARE QUALITY ATTRIBUTES	23
5 OTHER REQUIREMENTS	25
APPENDIX A – DATA DICTIONARY	26
APPENDIX B - GROUP LOG	27

Revisions

Version	Primary Author(s)	Description of Version	Date Completed
1.0	- Ana Torres - Diego Bernal - Javier Rodríguez	Versión inicial formal del documento SRS conforme al estándar IEEE. Incluye la definición completa del alcance y descripción de la solución tecnológica, requisitos funcionales y no funcionales y otros requerimientos. Esta versión consolida los comentarios recibidos durante la etapa de revisión preliminar y está lista para ser utilizada como base para el desarrollo de la solución tecnológica.	21/09/25
1.1	- Ana Torres - Diego Bernal - Javier Rodríguez - Guillermo Urbina	Versión corregida del documento de acuerdo a retroalimentación. Incluye casos de uso y diagramas de casos de uso, modificaciones en los puntos 3, 4 y 5.	28/09/25

1 Introduction

Este documento detalla la Especificación de Requisitos de Software (SRS) para el "Agente Inteligente para Agendar Citas Personales", versión 1.1. El proyecto se centra en el desarrollo de un sistema innovador que utiliza inteligencia artificial para ayudar a los usuarios a gestionar, agendar y organizar sus compromisos personales de manera eficiente e intuitiva.

Esta sección introductoria establece el marco del documento: define el propósito del producto, su alcance, el público objetivo, y presenta una visión general de cómo está organizado el contenido para facilitar su lectura y comprensión. Además, se incluyen las definiciones, acrónimos y convenciones de formato empleadas a lo largo del texto para garantizar la claridad y consistencia de la información.

1.1 Document Purpose

El propósito de este documento es definir y especificar de manera exhaustiva todos los requisitos para el desarrollo de la primera versión del software "Agente Inteligente para Agendar Citas Personales", identificado como versión 1.1. Este SRS está diseñado para servir como la fuente única de verdad para los equipos de desarrollo, pruebas y gestión de proyecto, estableciendo un acuerdo claro y unificado sobre las capacidades y el comportamiento esperado del producto final. Contiene una descripción detallada de los requisitos funcionales, no funcionales, las interfaces externas y las restricciones de diseño que guiarán su construcción y validación.

Este documento abarca el sistema completo en su totalidad, no un subsistema parcial. El alcance está enfocado en entregar un Producto Mínimo Viable (MVP) que valide las funcionalidades centrales del agente inteligente. Esto incluye la capacidad de registrar, consultar y modificar citas mediante comandos de voz y texto, la integración con calendarios externos (Google Calendar, Outlook, iOS) y el envío de recordatorios automáticos. El objetivo es establecer una base sólida y funcional que resuelva la necesidad principal del usuario, sobre la cual se podrán construir futuras mejoras y funcionalidades avanzadas.

1.2 Product Scope

El "Agente Inteligente para Agendar Citas Personales" es una aplicación de software diseñada para simplificar y automatizar la gestión de la agenda personal a través de la inteligencia artificial. Su propósito principal es actuar como un asistente proactivo que interpreta instrucciones en lenguaje natural (voz o texto) para organizar, recordar y centralizar los compromisos del usuario. Los beneficios clave para el usuario incluyen un ahorro significativo de tiempo, la reducción de la carga cognitiva asociada a la planificación y una disminución en los errores de agendamiento, como olvidos o conflictos de horario.

El objetivo del proyecto es ofrecer una herramienta centralizada e intuitiva que se integre de forma transparente en el ecosistema digital del usuario, conectándose con sus calendarios y asistentes virtuales existentes para enriquecer su funcionalidad en lugar de reemplazarla. Como meta principal para esta versión inicial (MVP), se busca validar la propuesta de valor del sistema, demostrando que la interacción mediante lenguaje natural es un método más eficiente y agradable para la organización de citas personales, mejorando así la productividad y el bienestar del usuario en su día a día.

1.3 Intended Audience and Document Overview

Este documento está dirigido a un conjunto diverso de lectores, cada uno con un interés específico en el proyecto. Los principales destinatarios son:

El Profesor: Como evaluador y supervisor del proyecto, utilizará este documento para valorar la correcta definición del alcance, la claridad de los requisitos y el cumplimiento de los objetivos académicos del curso de "Análisis, diseño y construcción de software".

El Cliente (conceptual): Representa a los futuros usuarios del sistema. Este SRS le permite entender qué hará el producto, cuáles son sus capacidades y limitaciones, y validar que la solución propuesta satisface sus necesidades de gestión de agenda.

El Equipo de Desarrollo (Equipo 21): Para los desarrolladores, este documento es la referencia técnica fundamental. Les proporciona la información detallada y necesaria para diseñar la arquitectura, desarrollar las funcionalidades y tomar decisiones de implementación que se alineen con los requisitos establecidos.

El Equipo de Pruebas (QA - Conceptual): Este grupo utilizará el SRS como base para diseñar y ejecutar los planes de prueba. Cada requisito aquí definido debe ser verificable, permitiéndoles validar que el software funciona según lo especificado y cumple con los criterios de calidad.

Para facilitar la lectura, el documento está organizado en las siguientes secciones principales: La **Sección 2** ofrece una descripción general del producto, su contexto, funcionalidades de alto nivel y las restricciones de diseño. La **Sección 3** profundiza en los requisitos específicos, detallando las interfaces de usuario y de sistema, así como todas las funcionalidades que el software debe realizar. La **Sección 4** describe los requisitos no funcionales, como el rendimiento, la seguridad y los atributos de calidad. Finalmente, la **Sección 5** cubre otros requerimientos, como la documentación y el soporte, y los **Apéndices** contienen información de apoyo como el diccionario de datos.

Se sugiere la siguiente secuencia de lectura según el tipo de audiencia:

Para el Profesor y el Cliente: Se recomienda una lectura enfocada en las Secciones **1, 2 y 4** para obtener una comprensión completa del propósito, el alcance general y los criterios de calidad del producto. Una revisión de los títulos de las funcionalidades en la **Sección 3** será útil para confirmar que todas las capacidades esperadas están cubiertas.

Para el Equipo de Desarrollo y Pruebas: Es indispensable la lectura completa y detallada de todo el documento. La **Sección 3** es de máxima importancia, ya que es la base para la implementación y la creación de casos de prueba. Las **Secciones 2 y 4** proporcionarán el contexto y las restricciones que influirán en el diseño técnico y los criterios de aceptación.

1.4 Definitions, Acronyms and Abbreviations

A continuación, se presenta una lista de los acrónimos y abreviaturas utilizados a lo largo de este documento para asegurar una interpretación consistente y precisa de los requisitos.

API (Application Programming Interface): Interfaz de Programación de Aplicaciones. Un conjunto de definiciones y protocolos que permite que dos aplicaciones de software se comuniquen entre sí. Se utiliza para la integración con calendarios y otros servicios externos.

COMET (Concurrent Object Modeling and Architectural Design Method): Metodología de diseño y modelado de software orientada a objetos para sistemas concurrentes y distribuidos.

GDPR (General Data Protection Regulation): Reglamento General de Protección de Datos. Normativa europea sobre la privacidad y protección de datos personales.

IA (Inteligencia Artificial): Campo de la informática dedicado a la creación de sistemas que pueden realizar tareas que normalmente requieren inteligencia humana, como el reconocimiento de voz o la toma de decisiones.

IEEE (Institute of Electrical and Electronics Engineers): Instituto de Ingenieros Eléctricos y Electrónicos. Organización profesional que, entre otras cosas, desarrolla estándares para la industria tecnológica.

iOS: Sistema operativo móvil desarrollado por Apple Inc. para sus dispositivos como el iPhone y el iPad.

LLM (Large Language Model): Modelo de Lenguaje Grande. Un tipo de modelo de inteligencia artificial entrenado con enormes cantidades de datos de texto para comprender, generar y responder a lenguaje humano de manera coherente.

MVP (Minimum Viable Product): Producto Mínimo Viable. Una versión inicial del producto que incluye solo las características esenciales para ser funcional y satisfacer a los primeros usuarios, permitiendo recoger retroalimentación para el desarrollo futuro.

NLP (Natural Language Processing): Procesamiento de Lenguaje Natural. Un área de la inteligencia artificial que se enfoca en la interacción entre las computadoras y el lenguaje humano, permitiendo a las máquinas entender e interpretar texto y voz.

OAuth (Open Authorization): Un estándar abierto para la delegación de acceso, comúnmente utilizado como una forma para que los usuarios de Internet otorguen a sitios web o aplicaciones acceso a su información en otros sitios web, pero sin darles las contraseñas.

QA (Quality Assurance): Aseguramiento de la Calidad. El conjunto de actividades y procesos sistemáticos implementados para garantizar que el software cumple con los requisitos de calidad especificados.

SAML (Security Assertion Markup Language): Un estándar abierto para el intercambio de datos de autenticación y autorización entre diferentes partes, en particular, entre un proveedor de identidad y un proveedor de servicios.

SSO (Single Sign-On): Inicio de Sesión Único. Un servicio de autenticación que permite a los usuarios utilizar un único conjunto de credenciales de inicio de sesión para acceder a múltiples aplicaciones.

SRS (Software Requirements Specification): Especificación de Requisitos de Software. Este mismo documento, que describe de manera formal el comportamiento y las funcionalidades de un sistema de software.

TLS (Transport Layer Security): Seguridad de la Capa de Transporte. Un protocolo criptográfico diseñado para proporcionar comunicaciones seguras a través de una red informática.

UML (Unified Modeling Language): Lenguaje de Modelado Unificado. Un lenguaje de modelado visual estandarizado que se utiliza para especificar, visualizar, construir y documentar los artefactos de un sistema de software.

1.5 Document Conventions

Este documento se adhiere a las convenciones de formato y estructura recomendadas por el estándar IEEE 830-1998 para la Especificación de Requisitos de Software, utilizando como base la plantilla proporcionada. El objetivo es mantener un formato claro, profesional y consistente que facilite su lectura y comprensión.

Las convenciones tipográficas específicas aplicadas son las siguientes:

Fuente: El cuerpo del texto está escrito en fuente Arial, tamaño 11.

Espaciado y Márgenes: Se utiliza interlineado sencillo y se mantienen márgenes de una pulgada (2.54 cm) en todos los lados de la página.

Estructura: La numeración y los títulos de las secciones y subsecciones siguen la jerarquía definida en la plantilla del documento.

No se emplean convenciones tipográficas adicionales, como colores o resaltados, que tengan un significado especial. Cualquier texto que aparezca en *cursiva* se utiliza para indicar comentarios o para dar énfasis, de acuerdo con las indicaciones de la plantilla original.

1.6 References and Acknowledgments

Este documento se ha desarrollado utilizando como referencia las siguientes fuentes y estándares, que han servido de guía para su estructura, contenido y para la definición de los requisitos del producto.

IEEE Recommended Practice for Software Requirements Specifications (IEEE Std 830-1998): Este estándar del Instituto de Ingenieros Eléctricos y Electrónicos ha sido la guía metodológica principal para la elaboración de este documento. Define la estructura, las características de un buen requisito y el contenido esencial que debe incluir una Especificación de Requisitos de Software de calidad.

Análisis del Estado del Arte en Asistentes de Agendamiento con IA: Para la definición de los requisitos y el alcance del proyecto, se realizó una investigación del panorama competitivo actual en el mercado de asistentes de agendamiento inteligente. El análisis de las

funcionalidades y estrategias de herramientas líderes como **Motion** y **Reclaim.ai** fue fundamental para establecer una propuesta de valor diferencial y definir un conjunto de características relevantes para el Producto Mínimo Viable (MVP).

Documentación de APIs de Servicios Externos: Aunque no se citan en su totalidad, el desarrollo del software dependerá de la consulta de la documentación técnica oficial para las APIs de los siguientes servicios con los que se integrará:

Google Calendar API: Documentación oficial disponible en <https://developers.google.com/workspace/calendar/api/guides/overview?hl=es-419> donde se describe la API REST para gestión de calendarios, eventos, reglas de acceso, configuraciones y más.

Microsoft Outlook Calendar API: La documentación se encuentra en el portal de Microsoft Graph, que es la plataforma para las APIs de Microsoft 365, incluyendo calendario.

Apple Push Notification Service (APNs): Documentación oficial en https://developer.apple.com/documentation/usernotifications/setting_up_a_remote_notification_server para notificaciones push en dispositivos iOS y macOS.

Firebase Cloud Messaging (FCM): Guías y referencias en <https://firebase.google.com/docs/cloud-messaging> para envío de mensajes y notificaciones a dispositivos móviles y web.

APIs de asistentes virtuales (Siri, Google Assistant, Alexa): Documentación oficial en las plataformas respectivas:

Siri: <https://developer.apple.com/siri/>

Google Assistant: <https://developers.google.com/assistant>

Alexa: <https://developer.amazon.com/en-US/alexa>

2 Overall Description

2.1 Product Overview

El Agente Inteligente para Agendar Citas Personales es un producto nuevo y autónomo que busca asistir a los usuarios en la gestión de su tiempo y compromisos. Surge de la necesidad de contar con una herramienta que centralice y automatice la organización de citas personales (médicas, reuniones sociales, trámites, etc.) de manera eficiente e intuitiva.

Este sistema no reemplaza directamente a otra aplicación, pero se integra con calendarios existentes (Google Calendar, Outlook) y asistentes virtuales, lo que permite aumentar su alcance. El producto forma parte de un ecosistema más amplio de aplicaciones de productividad, aunque funciona de manera independiente.

El sistema incluye un motor de IA que interpreta solicitudes en lenguaje natural, un módulo de calendario central, una interfaz gráfica amigable, y conectores con aplicaciones externas.

Diagrama del producto (funcionamiento general):

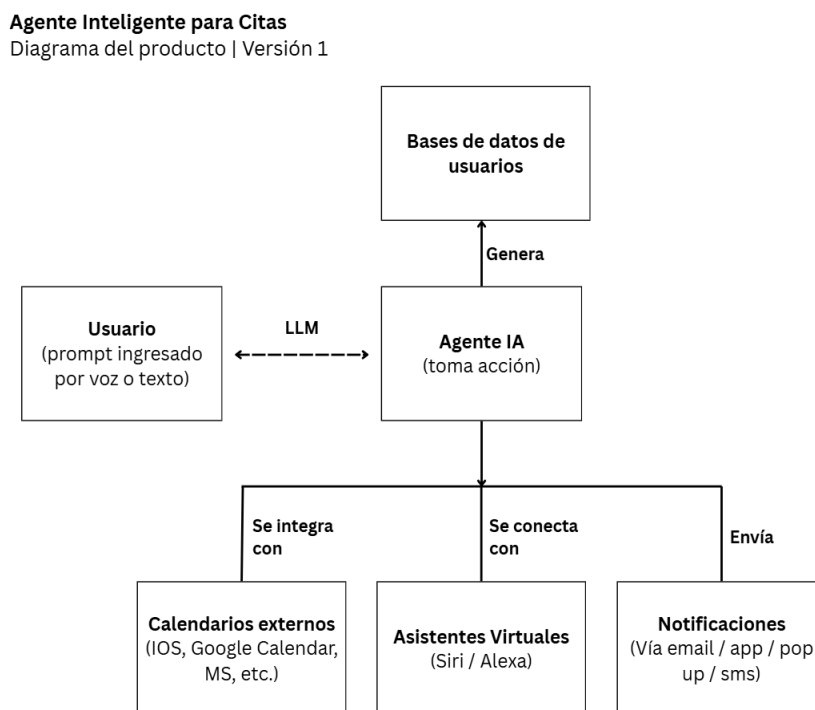


Figura 1: Diagrama general del funcionamiento del producto

2.2 Product Functionality

El sistema permitirá al usuario:

- Registrar citas mediante comandos de voz o texto.

- Consultar, modificar y cancelar citas.
- Integrarse con calendarios externos (Google, Outlook).
- Enviar recordatorios vía notificaciones móviles o correo electrónico.
- Gestionar citas recurrentes (por ejemplo, semanal, mensual).
- Detectar conflictos de horario y proponer alternativas.
- Aprender patrones de uso para sugerir horarios óptimos.
- Interactuar con asistentes virtuales (Siri, Alexa, Google Assistant).

2.3 Design and Implementation Constraints

- **Metodología de diseño:** Se utilizará **COMET** (Concurrent Object Modeling and Architectural Design Method) como marco de desarrollo de software.
- **Lenguaje de modelado:** Se empleará **UML (Unified Modeling Language)** para representar casos de uso, diagramas de clases y secuencia.
- **Limitaciones técnicas:**
 - Compatibilidad inicial con Android 10+, iOS 15+ y navegadores modernos.
 - Si el sistema se desarrolla buscando ser compatible con esos navegadores, podría tener limitantes al intentar ejecutarse en versiones anteriores.
 - Soporte inicial de hasta 100 usuarios concurrentes en MVP.
 - Procesamiento de lenguaje natural en español e inglés. Se iniciará con el desarrollo en español y, posteriormente, se desarrollará el soporte para inglés.
 - Requiere conexión a internet estable para sincronización de calendarios externos.
 - Cumplimiento con normativas de privacidad y protección de datos (ej. GDPR, leyes locales).

2.4 Assumptions and Dependencies

- Se asume que los usuarios tendrán conexión a internet estable para el funcionamiento en tiempo real.
- Se asume que los primeros usuarios utilizan el español como su lengua principal.
- El sistema dependerá de servicios externos de calendario (Google, Outlook); cualquier cambio en sus APIs podría afectar la integración.
- Se asume que el usuario proporcionará permisos para acceder a calendarios y enviar notificaciones.
- Se prevé la disponibilidad de servicios de nube (ej. AWS, Azure) para almacenamiento y procesamiento.
- El sistema podrá evolucionar con integraciones adicionales (ej. asistentes virtuales, herramientas corporativas).

3.1.2 Hardware Interfaces

El sistema interactuará con:

- Dispositivos móviles (smartphones, tablets).
- Micrófono (para entrada de voz).
- Notificaciones push en dispositivos móviles.
- Sensores de sistema operativo (ej. reloj, recordatorios).

3.1.3 Software Interfaces

- Frontend Sencillo (Aplicación de Usuario): Para el MVP, se desarrollará una aplicación cliente multiplataforma utilizando el framework Flutter.
- API Central: El núcleo será una API RESTful en Python que centralizará la lógica de negocio y la comunicación con la API de Gemini.
- Integraciones Externas (MVP y Futuro): El MVP se centrará en la API de Google Calendar usando OAuth 2.0, con una arquitectura modular para futuras expansiones (Outlook, etc.).
- Asistentes Virtuales (Visión a Futuro): La API se diseñará para permitir una futura integración con Siri, Google Assistant, etc.

3.2 Functional Requirements

3.2.1 F1: Interpretación y Creación Inteligente de Citas

El agente de IA, utilizando la API de Gemini, deberá analizar solicitudes en lenguaje natural (voz/texto) para extraer con precisión las entidades de una cita (título, fecha, hora, lugar) y registrar el evento en el calendario.

3.2.2 F2: Gestión Conversacional de la Agenda

El sistema permitirá al usuario interactuar con su agenda mediante un diálogo. El agente de IA deberá ser capaz de comprender y responder a consultas ("¿Qué tengo para mañana?"), así como ejecutar comandos de modificación o cancelación ("Mueve mi reunión de las 10 a las 11", "Cancela mi cita con el dentista").

3.2.3 F3: Programación de Recordatorios Contextuales

El agente de IA deberá interpretar solicitudes para crear recordatorios vinculados a eventos específicos (ej. "Recuérdame revisar el reporte una hora antes de la junta semanal"), programando la notificación correspondiente.

3.2.4 F4: Sincronización Transparente con Calendarios Externos

El sistema deberá actuar como un puente entre el agente de IA y los calendarios externos del usuario (Google Calendar, etc.). Todas las operaciones realizadas por el agente se sincronizarán de forma bidireccional y en tiempo real.

3.2.5 F5: Detección y Resolución Proactiva de Conflictos

Al recibir una solicitud que genera un conflicto de horario, el agente de IA deberá no solo detectarlo, sino también analizar la agenda y proponer de forma proactiva soluciones y horarios alternativos viables.

3.2.6 F6: Personalización y Adaptación al Usuario

El agente de IA utilizará un historial de interacciones y preferencias definidas por el usuario para personalizar sus respuestas. Esto incluye aprender horarios preferidos, ubicaciones frecuentes o la duración típica de ciertos tipos de eventos para hacer sugerencias más rápidas y precisas.

3.3 Use Case Model

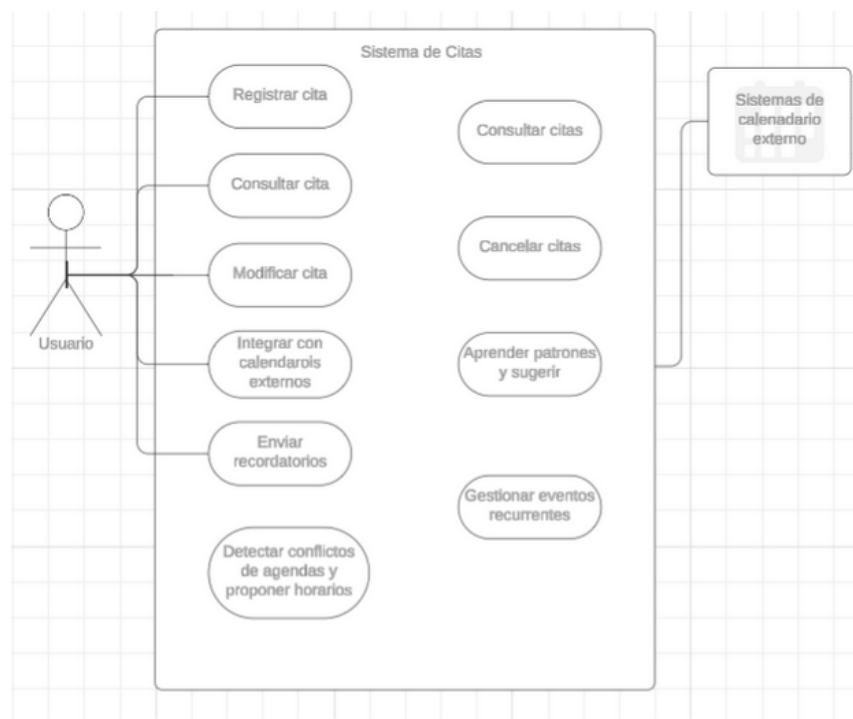


Figura 3: Diagrama de casos de uso del producto

3.3.1 Use Case #1 Registro de cita (UC1)

Proceso	Registro de cita mediante comando de lenguaje natural
Prioridad	Alta.
Proposito	Permitir al usuario agendar una cita en su sistema central y calendarios externos de

	manera rápida y eficiente a través de una instrucción de voz o texto.
Actores principales	Usuario.
Actores secundarios	Agente de IA (Sistema), API de Calendario Externo (Secundario).
Precondiciones	El usuario está autenticado en la aplicación y ha otorgado los permisos necesarios para acceder a su calendario.
Postcondiciones	La cita es creada y visible tanto en la aplicación como en el calendario externo sincronizado.
Flujo	1.- Se abre la aplicación. 2.- El usuario activa la interfaz de entrada y proporciona el comando: "Agenda una revisión dental para el próximo martes a las 4 PM en el consultorio". Si el comando es por voz, el sistema primero transcribe la instrucción a formato de texto. 3.- El Agente de IA (vía API de Gemini) analiza el comando y extrae las entidades: Título="Revisión dental", Fecha="próximo martes", Hora="4 PM", Lugar="consultorio". 4.- Se validan las condiciones del calendario como conflictos en agenda, etc. 5.- Se registra en el sistema central. 6.- Se sincroniza con calendarios externos. 7.- El sistema presenta una confirmación al usuario: "Confirmado. He agendado tu 'Revisión dental' para el [fecha] a las 4 PM".
Flujos secundarios	A1 (Conflicto de Horario): Si en el paso 3 se detecta un conflicto, el agente responde: "Veo que ya tienes 'Reunión de equipo' a esa hora. Te sugiero las 5 PM o el miércoles a la misma hora. ¿Cuál prefieres?" A2 (Fallo de Sincronización): Si en el paso 5 la API externa falla, el sistema marca la cita como "pendiente de sincronización" y reintenta el proceso automáticamente en segundo plano.
Excepciones en el flujo	Error con algún API externa, Lenguaje ambiguo en la solicitud textual o por voz, Conflictos de agenda, Conexión fallida.

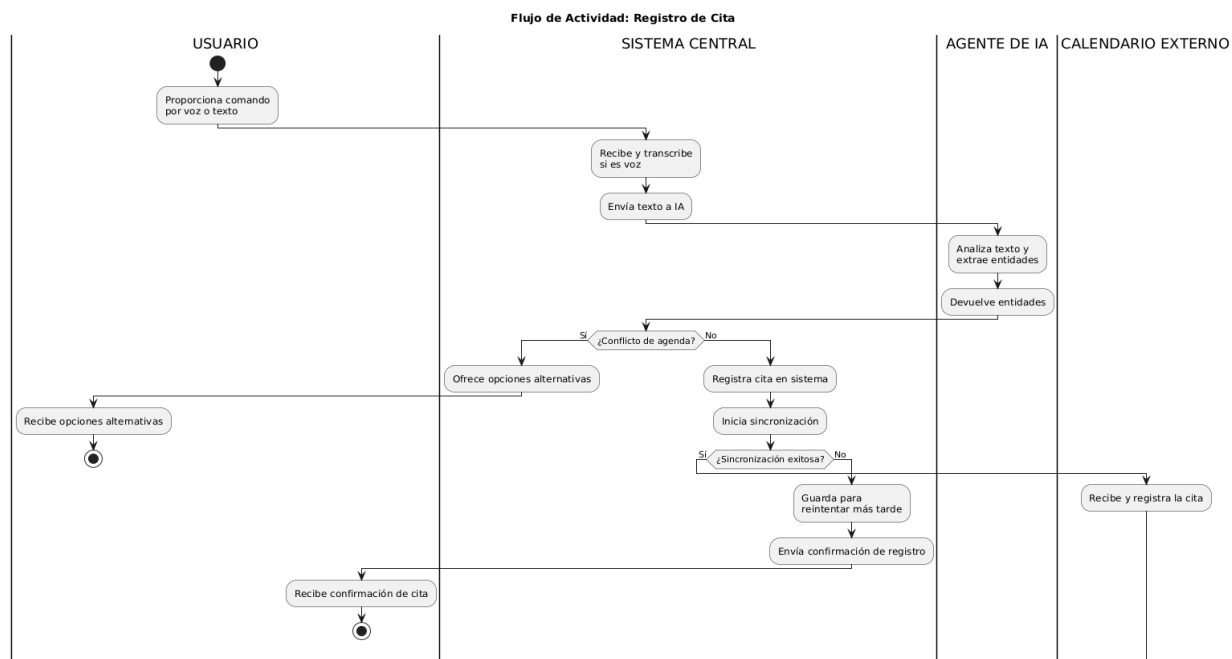


Figura 4: Diagrama del caso de uso 1 – Registro de cita

3.3.2 Use Case #2 Consulta y modificación de cita (UC2)

Proceso	Modificación de cita mediante comando conversacional.
Prioridad	Alta.
Proposito	Permitir al usuario consultar, modificar o eliminar citas ya registradas en el sistema centras y los calendarios externos.
Actores principales	Usuario.
Actores secundarios	Agente de IA (Sistema), API de Calendario Externo (Secundario)
Precondiciones	Se debe tener una cuenta activa. Deben existir citas ya registradas. Los calendarios deben estar sincronizados.
Postcondiciones	Las citas deben consultarse con la información correcta. Las citas modificadas deben estar actualizadas en el sistema centras y los calenadarios externos.
Flujo	1.- El usuario proporciona el comando: "Mueve mi reunión de mañana con el equipo de marketing a las 3 PM". Si el comando es por voz, el sistema primero transcribe la instrucción a formato de texto. 2.- El Agente de IA analiza el comando, identifica el evento de referencia ("reunión con equipo de marketing" de "mañana") y la

	acción solicitada (mover a "3 PM") 3.- El sistema verifica la disponibilidad para el nuevo horario (mañana a las 3 PM). No hay conflictos. 4.- El sistema ofrece las opciones para modificar o eliminar. 5.- Si el usuario selecciona modificar, ingresa los nuevos datos de la cita. 6.- El sistema valida los cambios. 7.- El sistema actualiza la cita en el sistema central. 8.- El sistema sincroniza los cambios con los calendarios externos. 9.- Si el usuario selecciona eliminar, el sistema solicita confirmación, elimina la cita y actualiza calendarios externos. 10.- El sistema confirma la acción al usuario.
Flujos secundarios	1.- Si no hay citas registradas, el sistema muestra un mensaje indicando que no hay información disponible. 2.- Si no hay conexión a Internet, las modificaciones o eliminaciones se guardan localmente y se sincronizan después.
Excepciones en el flujo	Error con algún API externa, Lenguaje ambiguo en la solicitud textual o por voz, Fallo en la cancelación o actualización, Conexión fallida.

	3.- El Agente de IA (vía API de Gemini) analiza la pregunta para interpretar la intención (consultar agenda) y el marco de tiempo solicitado ("mañana"). 4.- El sistema busca en su base de datos central todos los eventos que coincidan con el marco de tiempo especificado. 5.- El Agente de IA formatea los resultados encontrados en una respuesta de lenguaje natural. 6.- El sistema presenta la respuesta al usuario: "Mañana tienes dos eventos: 'Reunión de equipo' a las 10 AM y 'Revisión dental' a las 4 PM"
Flujos secundarios	A1 (Sin Eventos): Si en el paso 4 no se encuentra ningún evento, el agente responde: "Tu agenda para mañana está libre". A2 (Pregunta Ambigua): Si el usuario pregunta "¿Cuándo es la reunión?", y existen múltiples reuniones, el agente solicitará una clarificación: "Te refieres a la 'Reunión de equipo' o a la 'Reunión con Marketing'?".
Excepciones en el flujo	Error con algún API externa, Lenguaje ambiguo en la solicitud textual o por voz, Conflictos de agenda, Conexión fallida.

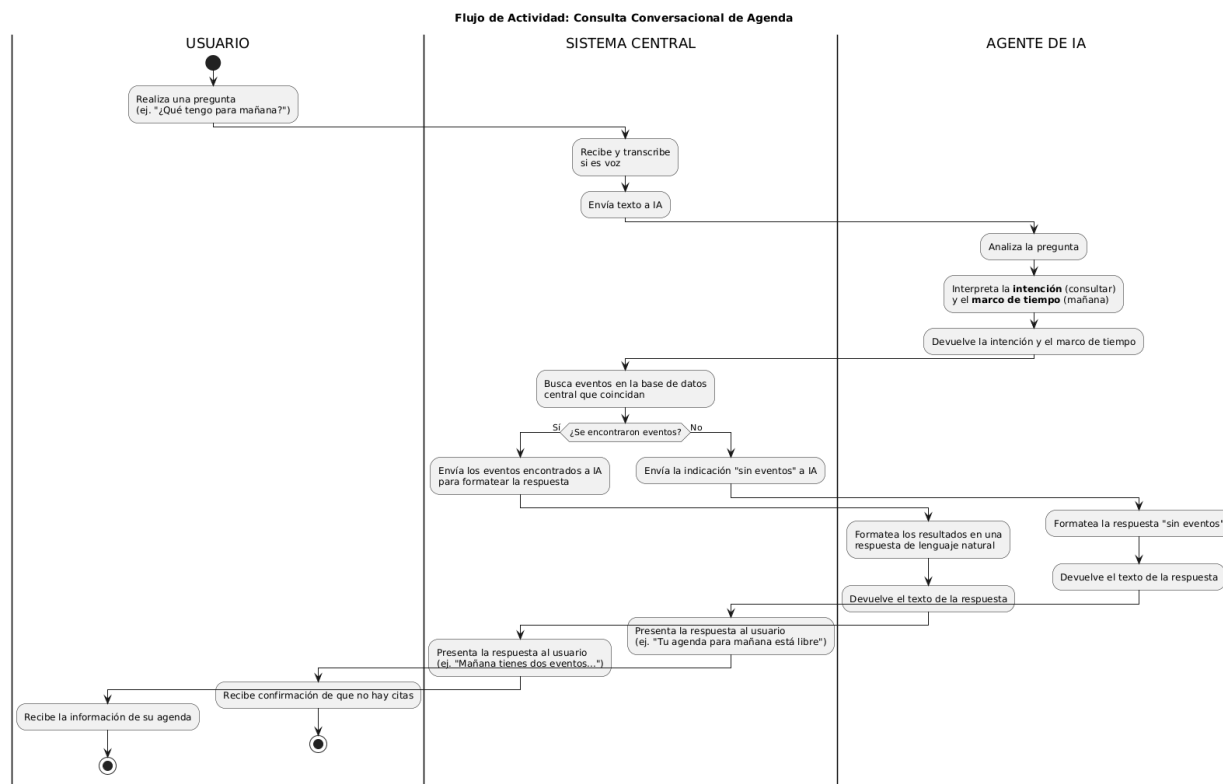


Figura 6: Diagrama del caso de uso 3 – Consulta de agenda de forma conversacional

3.3.4 Use Case #4 Crear un Recordatorio Contextual (UC4)

Proceso	Creación de recordatorio vinculado a un evento existente.
Prioridad	Media.
Proposito	Permitir al usuario programar notificaciones para tareas preparatorias o de seguimiento que están lógicamente asociadas a una cita específica en su calendario.
Actores principales	Usuario.
Actores secundarios	Agente de IA (Sistema), API de Calendario Externo (Secundario)
Precondiciones	El usuario está autenticado y el evento al que se quiere vincular el recordatorio ya existe en la agenda.
Postcondiciones	Se programa una nueva notificación en el sistema, la cual se activará en el momento especificado por el usuario.
Flujo	1.- El usuario proporciona un comando de voz o texto, como: "Recuérdame preparar la

	presentación una hora antes de la junta con el cliente". 2.- Si el comando es por voz, el sistema primero transcribe la instrucción a formato de texto. 3.- El Agente de IA analiza la solicitud e identifica tres componentes clave: la acción del recordatorio ("preparar la presentación"), el evento de anclaje ("junta con el cliente") y el desencadenante de tiempo ("una hora antes"). 4.- El sistema busca en la agenda el evento de anclaje ("junta con el cliente") para obtener su fecha y hora. 5.- El sistema calcula el momento exacto para la notificación (ej. si la junta es a las 11 AM, la notificación se programará para las 10 AM). 6.- El sistema guarda la nueva notificación en la base de datos, vinculándola al evento correspondiente. 7.- El sistema confirma la acción al usuario: "Entendido. Te recordaré que prepares la presentación una hora antes de tu junta con el cliente".
Flujos secundarios	A1 (Evento no Encontrado): Si en el paso 4 el sistema no puede localizar el "evento de anclaje" o encuentra múltiples coincidencias, el agente pedirá al usuario que especifique a qué evento se refiere.
Excepciones en el flujo	Error con algún API externa, Lenguaje ambiguo en la solicitud textual o por voz, Fallo en la cancelación o actualización, Conexión fallida.

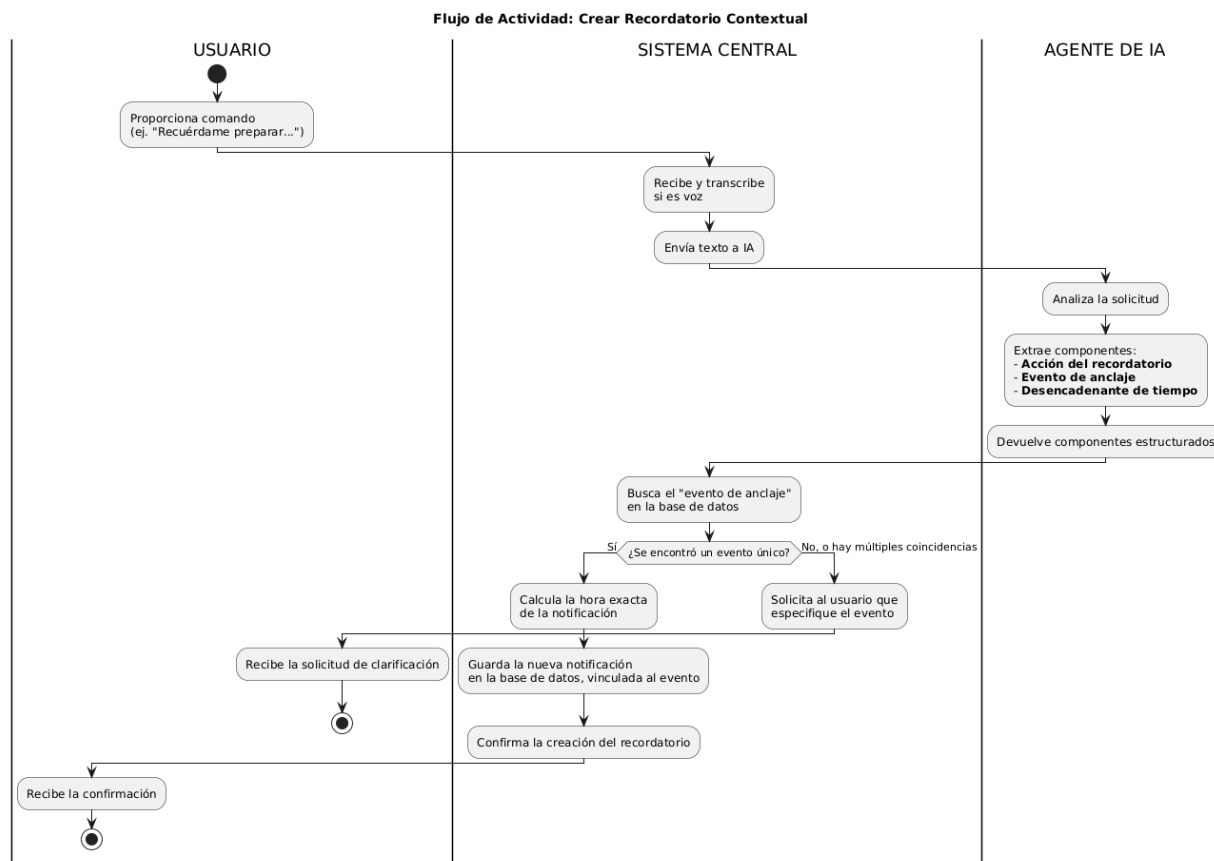


Figura 7: Diagrama del caso de uso 4 – Crear recordatorio contextual

3.3.5 Use Case #5: Agendar una Cita Recurrente (UC5)

Proceso	Creación de una serie de citas recurrentes.
Prioridad	Media.
Proposito	Permitir al usuario programar de manera eficiente eventos que se repiten en el tiempo (ej. reuniones semanales, clases, recordatorios mensuales) mediante un único comando en lenguaje natural.
Actores principales	Usuario.
Actores secundarios	Agente de IA (Sistema), API de Calendario Externo (Secundario).
Precondiciones	El usuario está autenticado en la aplicación.
Postcondiciones	Una serie de eventos vinculados es creada en el sistema central y sincronizada con los

	calendarios externos, siguiendo la regla de recurrencia especificada.
Flujo	1.- El usuario proporciona un comando de voz o texto, como: "Agenda una clase de yoga todos los martes y jueves a las 7 AM, empezando la próxima semana". 2.- Si el comando es por voz, el sistema primero transcribe la instrucción a formato de texto. 3.- El Agente de IA (vía API de Gemini) analiza el comando y extrae no solo las entidades básicas (título, hora), sino también la regla de recurrencia (frecuencia: semanal; días: martes, jueves; fecha de inicio: próxima semana). 4.- El agente de IA detecta que falta una fecha de finalización y solicita una clarificación: "Entendido. ¿Hasta cuándo debo programar las clases de yoga?". 5.- El usuario responde: "Hasta finales de marzo". 6.- El sistema verifica si existen conflictos para la primera o las primeras ocurrencias de la serie para alertar al usuario de posibles problemas iniciales. 7.- El sistema crea el evento maestro con la regla de recurrencia completa en la base de datos central y lo sincroniza con la API del calendario externo. 8.- El sistema confirma la acción al usuario: "Hecho. He agendado tus clases de yoga todos los martes y jueves a las 7 AM hasta finales de marzo".
Flujos secundarios	A1 (Conflicto en la Serie): Si en el paso 6 se detecta un conflicto, el agente informará: "Veo un conflicto para el primer jueves. ¿Quieres omitir esa fecha o busco un nuevo horario para toda la serie?". A2 (Regla Implícita): Si el usuario no especifica una fecha de finalización y no responde a la clarificación, el sistema puede aplicar una regla por defecto (ej. "programar por los próximos 3 meses") e informar al usuario de esta suposición.
Excepciones en el flujo	Error con algún API externa, Lenguaje ambiguo en la solicitud textual o por voz, Conflictos de agenda, Conexión fallida.

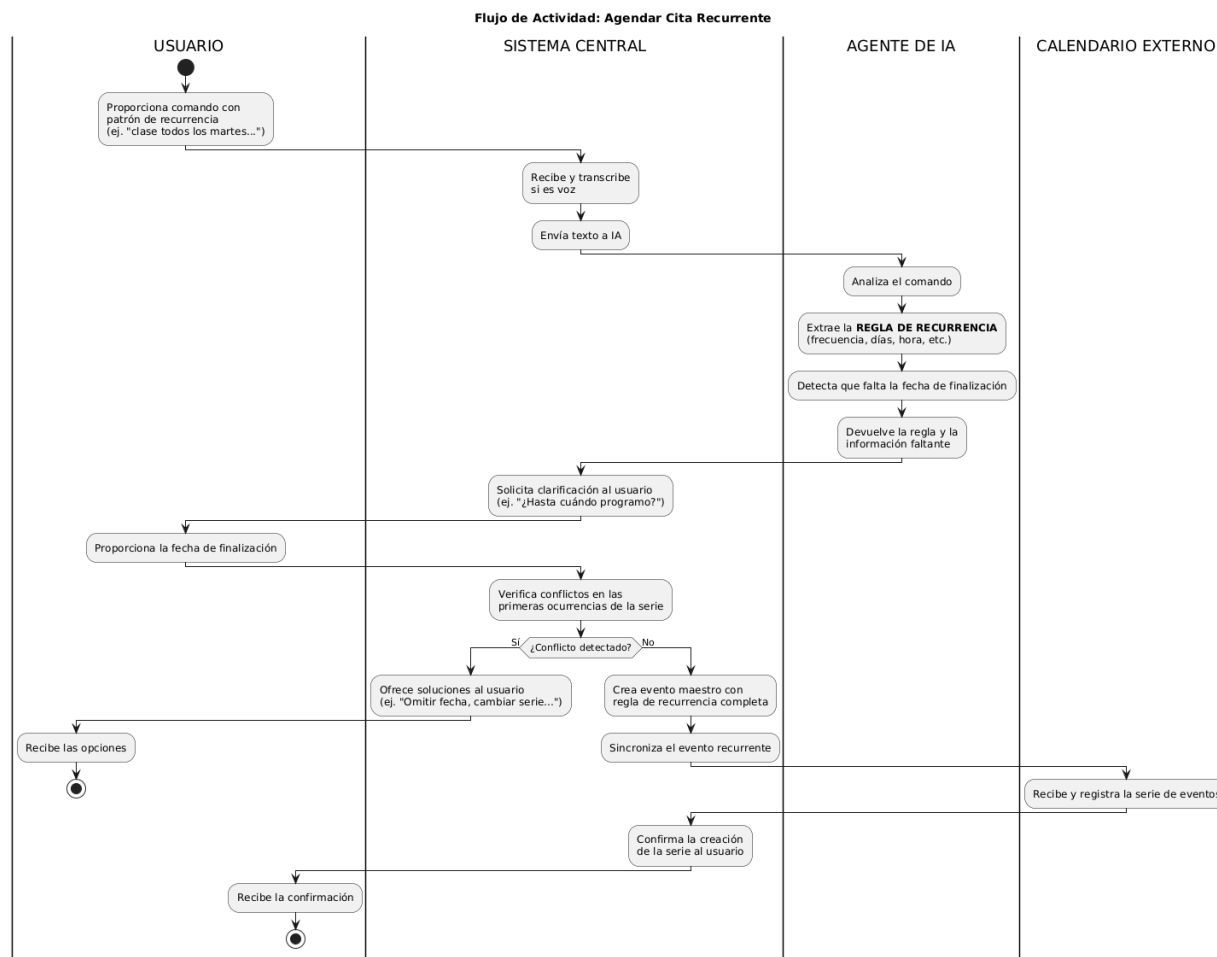


Figura 8: Diagrama del caso de uso 5 – agendar una cita recurrente

3.3.6 Use Case #6 Configurar Preferencias del Asistente (UC6)

Proceso	Establecer reglas y preferencias personales para el agendamiento.
Prioridad	Media.
Proposito	Permitir al usuario enseñar explícitamente al agente de IA sus reglas y hábitos de programación para que las futuras sugerencias y acciones sean más inteligentes y personalizadas (Requerimiento F6).
Actores principales	Usuario.

Actores secundarios	Agente de IA (Sistema), API de Calendario Externo (Secundario)
Precondiciones	El usuario está autenticado.
Postcondiciones	Las preferencias del usuario son almacenadas en su perfil dentro del sistema y serán utilizadas por el agente de IA en futuras interacciones.
Flujo	1.- El usuario navega a la sección de configuración o proporciona un comando directo: "Quiero configurar mis preferencias". 2.- El usuario proporciona una regla en lenguaje natural: "No me agendes reuniones los viernes después de las 2 PM". 3.- Si el comando es por voz, el sistema primero transcribe la instrucción a formato de texto. 4.- El Agente de IA analiza la instrucción y la traduce a una regla estructurada (ej. Regla: No Agendar; Día: Viernes; Horario: >14:00). 5.- El sistema almacena esta regla en el perfil del usuario en la base de datos. 6.- El sistema confirma que la preferencia ha sido guardada: "Entendido. A partir de ahora, evitaré agendarte reuniones los viernes después de las 2 de la tarde".
Flujos secundarios	A1 (Modificar Preferencia): El usuario puede decir: "Cambia mi regla de los viernes, ahora que no sea después de las 4 PM". El agente localizará la regla existente y la actualizará. A2 (Aplicación de la Regla): En un futuro caso de uso de creación de citas, si alguien intenta agendar una reunión con el usuario un viernes a las 3 PM, el agente usará esta regla para rechazar automáticamente o proponer un horario alternativo que la cumpla.
Excepciones en el flujo	Error con algún API externa, Lenguaje ambiguo en la solicitud textual o por voz,

Fallo en la cancelación o actualización,
Conexión fallida.

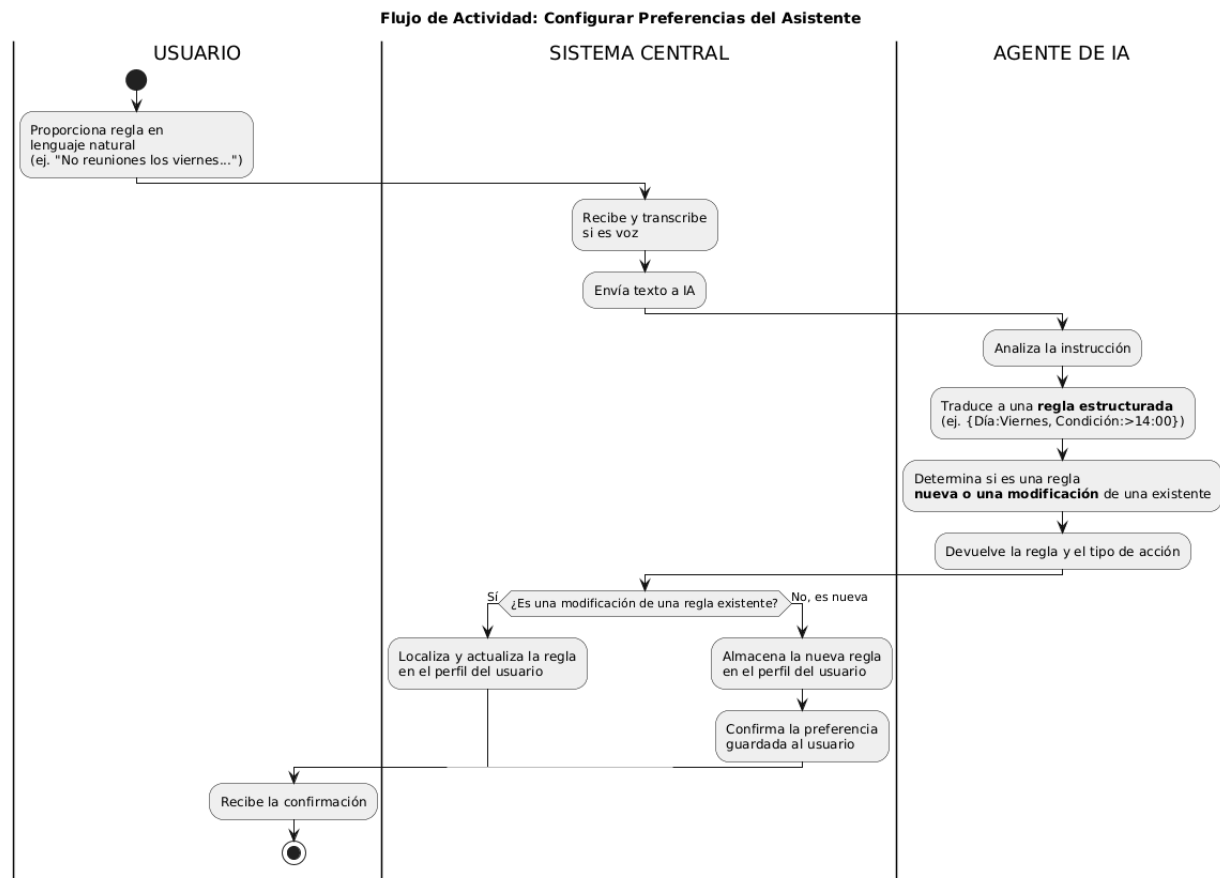


Figura 9: Diagrama del caso de uso 6 – Configurar preferencias del asistente

4 Other Non-functional Requirements

4.1 Performance Requirements

La solución tecnológica debe estar diseñada para ofrecer una experiencia ágil, respondiendo al 90% de las solicitudes en menos de 2 segundos bajo condiciones normales. Además, podrá atender al menos 100 usuarios conectados simultáneamente, sin afectar su funcionamiento en el primer MVP, con una arquitectura escalable que permita soportar hasta 1.000 usuarios en futuras versiones, conforme crezca las necesidades del proyecto.

4.2 Safety and Security Requirements

La solución debe proteger adecuadamente la información de los usuarios y el funcionamiento del sistema. En la primera versión, el acceso se controlará mediante un sistema de usuario y contraseña guardado de forma segura. En versiones futuras, se integrará una solución de inicio de sesión único (SSO de mercado) que permitirá a los usuarios autenticarse de manera más sencilla y segura, utilizando estándares reconocidos como OAuth, OpenID Connect o SAML.

Toda la información en tránsito, que viaje entre la plataforma y los usuarios estará encriptada con TLS 1.3 para evitar que sea interceptada, y las credenciales y datos sensibles se almacenarán también de forma protegida.

La solución tecnológica será sometida a análisis estático de código antes de cada liberación, así como análisis dinámico de seguridad en entornos de preproducción (como pruebas de penetración). Todos los hallazgos críticos deberán ser corregidos antes de la puesta en producción.

4.3 Software Quality Attributes

La solución tecnológica estará diseñada para ofrecer una experiencia de uso confiable, accesible y adaptable, cumpliendo con estándares de calidad que garanticen su funcionamiento eficiente y seguro. Esto se logrará a través de los siguientes mecanismos y principios de diseño:

4.3.1 Usabilidad y Experiencia de Usuario

La facilidad de uso será el pilar del sistema, lograda a través de una estrategia de diseño conversacional-céntrico.

- **Mecanismo Principal:** Se priorizará una interfaz minimalista, donde la interacción principal será un campo de entrada de texto/voz, similar a una aplicación de mensajería. Este paradigma es instantáneamente familiar para el usuario y reduce la carga cognitiva al eliminar menús complejos para las acciones primarias.
- **Coherencia y Estándares:** La interfaz se adherirá estrictamente a las guías de diseño de cada plataforma: Material Design de Google para Android y la web, y Human Interface Guidelines de Apple para iOS. Esto asegura que los controles, las animaciones y la navegación se sientan nativos y predecibles para el usuario.

- **Feedback Constante:** El agente de IA proporcionará retroalimentación inmediata y clara para cada acción (confirmaciones, solicitudes de clarificación, mensajes de error), evitando que el usuario se sienta inseguro sobre el estado del sistema.
- **Validación con Usuarios:** Se realizarán pruebas de usabilidad iterativas durante el desarrollo, utilizando el método de "pensamiento en voz alta" con los perfiles de usuario definidos (User Personas) para identificar y corregir puntos de fricción antes del lanzamiento.

4.3.2 Accesibilidad

Se garantizará que la aplicación sea utilizable por el mayor número posible de personas, incluyendo aquellas con discapacidades, mediante la implementación de las siguientes características:

- **Compatibilidad con Lectores de Pantalla:** Todos los elementos interactivos de la interfaz (botones, campos de texto, etc.) serán correctamente etiquetados para ser compatibles con **VoiceOver (iOS)** y **TalkBack (Android)**, permitiendo a los usuarios con discapacidad visual navegar y operar la aplicación.
- **Contraste y Tipografía Adaptable:** El diseño de la interfaz cumplirá con los ratios de contraste de nivel **AA de las WCAG (Web Content Accessibility Guidelines)**. Además, la aplicación respetará el tamaño de fuente definido por el usuario en la configuración de su sistema operativo.
- **Zonas de Interacción Claras:** Todos los botones y elementos táctiles tendrán un tamaño mínimo de 44x44 puntos para asegurar que sean fáciles de presionar, beneficiando a usuarios con discapacidades motoras.

4.3.3 Adaptabilidad y Mantenibilidad

La capacidad del sistema para evolucionar y ser mantenido a largo plazo se asegurará a través de decisiones arquitectónicas clave:

- **Arquitectura Modular:** El sistema se construirá sobre una arquitectura de tres capas desacopladas:
 1. **Frontend (Flutter):** Responsable únicamente de la interfaz de usuario.
 2. **Backend (API en Python):** Contiene toda la lógica de negocio y la orquestación con la IA.
 3. **Capa de Servicios Externos:** Módulos específicos para conectar con cada API externa (Google Calendar, etc.). Esta separación permite que se pueda actualizar o incluso reemplazar una capa (ej. añadir una nueva integración de calendario) sin afectar al resto del sistema.
- **Diseño Basado en API (API-First):** La API RESTful actuará como un "contrato" formal entre el frontend y el backend. Esto facilita el desarrollo en paralelo y permite que en el futuro se puedan crear nuevos clientes (ej. una aplicación de escritorio o un plugin) que consuman la misma lógica de negocio sin necesidad de reescribirla.

- **Gestión de Configuración:** Todas las claves de API, credenciales y configuraciones específicas del entorno se gestionarán a través de variables de entorno, no estarán codificadas en el código fuente. Esto permite adaptar y desplegar la aplicación en diferentes entornos (desarrollo, pruebas, producción) de manera segura y eficiente.

5 Other Requirements

Mantenimiento y Soporte

La solución tecnológica se entregará con una documentación técnica exhaustiva, que incluirá diagramas de arquitectura, una guía detallada de la API RESTful y manuales de despliegue para facilitar futuras actualizaciones y la incorporación de nuevos desarrolladores al proyecto.

El mantenimiento del sistema incluirá actividades proactivas como el monitoreo del rendimiento de los servidores, la actualización periódica de las librerías y dependencias de software para mitigar vulnerabilidades de seguridad, y la realización de copias de seguridad automáticas y encriptadas de la base de datos para garantizar la recuperación ante desastres.

Para el usuario final, se ofrecerá soporte a través de una mesa de ayuda accesible por correo electrónico durante el horario laboral estándar (9 AM - 5 PM, hora local). Esta mesa de ayuda deberá cumplir con un requisito de tiempo de primera respuesta inferior a 24 horas hábiles y será responsable de guiar a los usuarios en la resolución de problemas comunes, gestionar reportes de errores y escalar incidencias técnicas críticas al equipo de desarrollo.

Escalabilidad y Evolución Futura

La arquitectura del sistema está diseñada explícitamente para prever la futura incorporación de funcionalidades inteligentes y la integración con un ecosistema más amplio. La arquitectura modular y basada en API es la acción fundamental que se realiza para asegurar esta evolución.

Al centralizar toda la lógica en una API robusta, se facilita enormemente la conexión con nuevos servicios externos. La integración con asistentes virtuales como Alexa, Siri o Google Assistant, por ejemplo, se realizará creando una capa de "adaptadores" que traducirán las solicitudes de las plataformas de estos asistentes (Alexa Skills Kit, SiriKit) a llamadas a nuestra API existente, sin necesidad de modificar el núcleo del sistema.

De manera similar, la incorporación de funciones de aprendizaje más avanzadas, como el análisis predictivo de la carga de trabajo del usuario o la sugerencia proactiva de bloques de tiempo para descanso, se implementará como nuevos microservicios dentro del backend que enriquecerán las respuestas del agente de IA, asegurando que el producto pueda crecer en inteligencia y alcance de forma sostenible.

Appendix A – Data Dictionary

El presente Diccionario de Datos sirve como un repositorio centralizado que proporciona definiciones formales para todos los elementos de datos clave utilizados en el sistema "Agente Inteligente para Agendar Citas Personales". Su propósito fundamental es eliminar la ambigüedad y garantizar un entendimiento común y consistente de las estructuras de datos entre los equipos de desarrollo, pruebas y gestión del proyecto.

La siguiente tabla detalla las entidades principales del sistema, como Cita y Usuario, especificando su descripción, tipo de dato, formato y, crucialmente, su trazabilidad directa con los requisitos funcionales descritos en la Sección 3.2. Este diccionario es un documento vivo y será actualizado continuamente a medida que el diseño del sistema se refine y evolucionen los requerimientos.

Nombre del Elemento	Descripción	Tipo de Dato	Posibles Valores/Formato	Operaciones Relacionadas	Requerimiento(s) Asociado(s)
Cita	Objeto que representa un evento en el calendario.	Objeto/Estructura	Contiene: titulo, fecha, hora_inicio, hora_fin, lugar, descripcion.	Crear, Leer, Actualizar, Eliminar (CRUD)	F1, F2, F5
Cita.titulo	El nombre o motivo de la cita.	Cadena de texto	Máx 255 caracteres.	Crear, Actualizar	F1
Usuario	Representa al usuario del sistema.	Objeto/Estructura	Contiene: id_usuario, nombre, email, preferencias_horario.	Autenticar, Actualizar Perfil	F6
Notificacion	Un recordatorio enviado al usuario.	Objeto/Estructura	Contiene: id_cita, fecha_envio, canal (email, push).	Enviar Recordatorio	F3
ConflictoHorario	Estado que indica una superposición de citas.	Booleano	true, false	Detectar Conflicto	F5

Appendix B - Group Log

Reunión 1: Definición Inicial del Proyecto y Asignación de Tareas del SRS

Fecha: 17 de septiembre de 2025

Asistentes: Ana Cristina Torres, Diego Bernal, Javier Rodríguez

Minuta:

Se discutió el alcance del proyecto "Agente Inteligente para Agendar Citas Personales", enfocándolo en un Producto Mínimo Viable (MVP).

Se definieron los 6 requerimientos funcionales principales (F1-F6) que formarán el núcleo del sistema.

Se revisó la plantilla del SRS y se distribuyeron las secciones iniciales.

Acuerdos: Primera versión consolidada para revisión interna el 20 de septiembre.